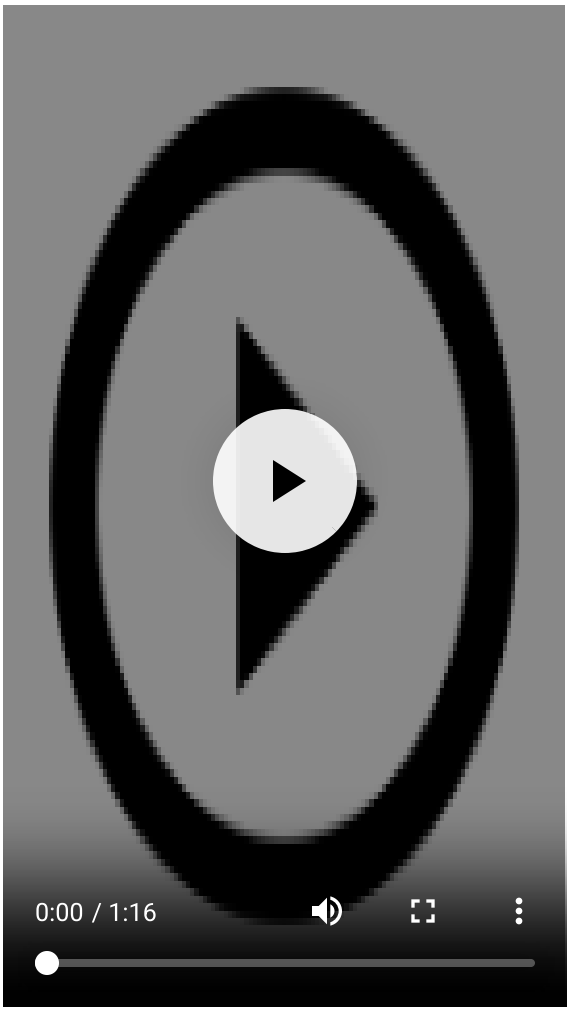
Activity 4.3: Develop the ASP.NET Web Application



By the end of this activity, your group should have a working ASP.NET web application ready for integration with Flutter in Activity 4.4.

Supporting Video: Building an ASP.NET Web Application

Watch the following tutorial to understand how an ASP.NET Core MVC application is developed:

Build a CRUD App with ASP.NET Core MVC (.NET 9 + EF Core Tutorial) - YouTube . (https://www.youtube.com/watch?v=FeGXk8GagFg)

While watching, focus on:

Topic	What to learn
Project setup	How an ASP.NET Core MVC project is created
Models	How application data is represented
Controllers and views	How application functions and pages are organised
CRUD operations	How records are created, retrieved, updated and deleted
Forms and validation	How users submit information and receive feedback

Important: The video demonstrates SQL Server and Entity Framework Core. Your project must continue using the shared Supabase backend developed in Activity 4.1. Do not create a separate SQL Server database simply to follow the video.

For the Supabase connection, refer to the C#: Introduction | Supabase Docs (https://supabase.com/docs/reference/csharp/introduction) . This documentation describes the community-maintained C# client for accessing Supabase services.

Activity at a Glance

You will develop	Main outcome
ASP.NET project structure	Organised web application code
Approved web pages	Working web interfaces
Forms and navigation	Users can complete the intended processes
Required web functions	Features based on the SRS
Supabase connection	Web application uses the shared backend
Initial function checks	Evidence of working web features

What You Must Do

Step 1: Review the Approved Web Application Requirements

Open your updated SRS and review the web interface designs, use case diagram, sequence diagrams and relevant requirements.

Identify the functions assigned to the ASP.NET web application.

Complete the following table for your own project.

Web function	Related requirement or use case	Required page

Short example: Salon appointment system

Web function	Related use case	Required page
Staff sign-in	Login	Login page
View requests	View Appointment Requests	Appointment list
Approve or reject a request	Manage Appointment	Appointment details
View appointment information	View Appointments	Dashboard

The functions for your project must reflect your stakeholder's approved requirements.

Step 2: Prepare the ASP.NET Project

Open your group's existing ASP.NET project, or create it if the project has not yet been prepared.

Use the ASP.NET approach selected for your approved design, such as ASP.NET Core MVC.

If using MVC, organise the project appropriately:

Project component	Purpose
Models	Represent the information used by the application
Views	Display the web pages
Controllers	Handle requests and coordinate application functions
Services	Organise communication with Supabase and other required services

Keep the web application in the web/ folder of your existing GitHub repository.

Check that the application launches successfully before implementing the remaining functions.

For additional guidance on project setup and MVC structure, use the Get started with ASP.NET Core MVC | Microsoft Learn (https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-mvc-app/start-mvc?view=aspnetcore-10.0&tabs=visual-studio) .

Step 3: Develop the Approved Web Pages

Use the wireframes developed in Unit 3 to implement the web application interfaces.

Identify the information, buttons, forms and tables required on each page.

For example:

Page	**Possible interface elements**
Login	Email, password and sign-in button
Dashboard	Summary of relevant information
Appointment list	Appointment details and current status
Appointment details	Approve and Reject buttons
Report page	Relevant information for authorised users

Your web application must be appropriate for its intended users. Do not add functions that are outside the approved project scope simply because they appear in an online tutorial.

Step 4: Implement Navigation and Forms

Connect the web pages so users can move through the application.

Implement the required forms and input validation.

Short example: A staff member opens the appointment list, selects a request, reviews its details and approves or rejects it.

The application must provide clear feedback when an operation succeeds or fails.

Use your SRS to determine which functions should be accessible to each user role.

Step 5: Connect ASP.NET to the Shared Supabase Backend

Connect the ASP.NET application to the same Supabase project configured in Activity 4.1 and used by Flutter in Activity 4.2.

Implement the database operations needed by the web application.

Operation	**Salon example**

Create	Add a record where the staff role permits it
Read	Retrieve appointment requests
Update	Approve or reject an appointment
Delete	Remove a record only where permitted by the SRS

Use an appropriate Supabase integration approach for your ASP.NET project. Keep sensitive credentials in secure server-side configuration, not in browser code or the GitHub repository.

Where users sign in, ensure the backend applies the required permissions to their requests. Do not use unrestricted administrative access as a substitute for implementing the approved user roles.

Step 6: Implement the Main Web Functions

Develop the functions identified in Step 1.

For each function, confirm that the relevant user can complete the intended process.

Short example: Approving an appointment

1. Staff sign in to the ASP.NET web application.
2. The application retrieves the appointment requests from Supabase.
3. Staff select a request and choose Approve.
4. The application updates the appointment status in Supabase.
5. The updated status is displayed in the web application.

In Activity 4.4, the group will demonstrate that the same status can be retrieved through the Flutter application.

Step 7: Perform Initial Function Checks

Use suitable dummy data to check the implemented web functions.

Function or check	**Result**	**Action required**
Web application launches successfully		
Main pages display correctly		
Navigation works		
Forms validate user input		
Web application connects to Supabase		

Required records can be retrieved		
Records can be created or updated where required		
Authentication works, where required		
User access restrictions work as intended		

Correct the problems identified during development. Formal testing of the complete prototype will take place in Unit 5.

Step 8: Commit Your Work and Update Project Records

Commit your web application code to the group's existing GitHub repository.

Update the following:

Document or tool	**What to update**
SRS	Record any approved changes to the requirements or design
GitHub	Commit web application code and maintain relevant documentation
Microsoft Project	Record completed web development tasks and current progress
Project Progress Tracker	Record completed work, individual contributions and challenges

Do not start a new SRS or a separate repository for Phase 3.

What to Keep as Evidence

Evidence	**What it should show**
Web application screenshots	Implemented pages and navigation
Function screenshots or short recordings	Main web functions working

Supabase evidence	Required information retrieved or updated through ASP.NET
GitHub commits	Development progress and contributions
Updated Microsoft Project	Current development progress
Updated Project Progress Tracker	Completed tasks, contributions and challenges

Do not include passwords, secret keys or confidential personal information in your evidence.

Activity Outcome

A working ASP.NET web application with the approved pages, navigation and main functions implemented.

The application must connect to the shared Supabase backend and be ready for integration with the Flutter mobile application.

Before Moving On

Confirm that your group has:

- implemented the approved web pages and navigation;
- developed the required forms and web functions;
- connected ASP.NET to the shared Supabase backend;
- checked the main functions using suitable test data;
- retained evidence of the working web application;
- committed relevant code to GitHub; and
- updated Microsoft Project and the Project Progress Tracker.

Next: Activity 4.4: Integrate the Mobile and Web Applications.